

Multi-Method Approach to Training a Competitive No-Limit Texas Hold'em Poker Agent Using MCTS, CFR, and ReBeL Architectures

Miles Vollmer, Dhananjai Senthil Kumar

Northeastern University, Reinforcement Learning 4180, Chris Amato
vollmer.m@northeastern.edu, senthilkumar.d@northeastern.edu

Abstract

Multiplayer imperfect-information games, such as 6-player No-Limit Texas Hold'em poker, pose significant challenges for reinforcement learning (RL) algorithms due to partial observability, dynamic interactions, and complex strategic behaviors. In this paper, we explore two cutting-edge algorithms, Recursive Belief-based Learning (ReBeL) and Monte Carlo Tree Search (MCTS) with Counterfactual Regret Minimization (CFR), to address these challenges. Each algorithm is analyzed and extended to a 6-player context, focusing on their applicability and performance in the complex environment of multiplayer Texas Hold'em.

We evaluate both algorithms through experiments playing against a variety of different agents in a 6-player poker setting, highlighting their respective strengths and trade-offs in terms of strategic adaptability, as well as showing how well they can adapt and figure out the strategies of their opponents. Our findings show that ReBeL clearly outperforms Q-Learning but can struggle against aggressive deterministic agents.

Introduction

Poker AI has evolved significantly over the years, with breakthroughs such as DeepStack, Libratus, and Pluribus demonstrating the feasibility of solving increasingly complex poker variants. Early systems relied heavily on simplified abstractions of the game, but more recent advancements leverage reinforcement learning (RL) and search techniques to achieve near-optimal strategies in two-player Heads-Up No-Limit Texas Hold'em.

Despite these advancements, No-Limit Texas Hold'em remains a highly challenging domain. Unlike two-player games, multiplayer variants introduce additional layers of complexity, including multi-agent interactions, larger strategy spaces, and increased variability in outcomes. These factors make it difficult to develop effective algorithms capable of generalizing across diverse scenarios.

Additionally, practical implementation challenges compound these issues. For example, during our experiments, we faced significant limitations with the available Python environments for Texas Hold'em. The implementation we

chose required extensive debugging to address pot distribution issues, which remain imperfect and may introduce noise into the experimental results.

Several advanced algorithms have been proposed to tackle the complexities of poker:

- ReBeL: Combines reinforcement learning with search to handle imperfect-information games effectively.
- DeepStack: Utilizes continual re-solving and depth-limited search for two-player No-Limit Hold'em.
- Pluribus: The first AI capable of beating professional players in multiplayer poker.
- Counterfactual Regret Minimization (CFR): A foundational algorithm for solving extensive-form games.

While these methods have achieved remarkable success in two-player games, they often struggle to scale effectively to multiplayer settings. Strategies optimized for two-player interactions fail to account for the added dynamics introduced by additional players.

Motivation

The development of more robust poker agents remains an ongoing challenge, particularly in multiplayer No-Limit Texas Hold'em, where the ability to read opponents and adapt tactically is crucial. Existing algorithms, while effective in limited scenarios, often lack the flexibility to handle the dynamic and unpredictable nature of multiplayer games. Addressing these shortcomings requires innovative approaches that can blend the strengths of multiple reinforcement learning (RL) techniques.

Combining Counterfactual Regret Minimization (CFR) with advanced algorithms such as Recursive Belief-based Learning (ReBeL) and Monte Carlo Tree Search (MCTS) provides a promising path forward. CFR excels at minimizing regret over extensive-form games, while ReBeL and MCTS offer powerful frameworks for dealing with imperfect information and decision-making under uncertainty. Integrating these methods allows for a more granular breakdown of imperfect information, enabling agents to better utilize concepts such as public belief states and exploit strategic opportunities in complex environments.

Beyond poker, the insights gained from this approach have the potential to benefit other imperfect-information

games. Regret minimization and belief-state modeling are generalizable concepts that could extend to domains like real-time strategy games, negotiation scenarios, or financial modeling.

Background

Monte Carlo Tree Search (MCTS)

Monte Carlo Tree Search (MCTS) is a popular algorithm used in decision-making problems involving large state and action spaces. It balances exploration and exploitation by iteratively simulating game play to estimate action values. The core mechanics of MCTS consist of four stages:

- (i) **Selection:** Traverse the tree from the root using a selection policy (e.g., Upper Confidence Bound) to choose the most promising node.
- (ii) **Expansion:** Add one or more new nodes to the tree when an unexplored action is reached.
- (iii) **Simulation:** Run random (or policy-guided) simulations from the newly expanded node to terminal states, estimating outcome values.
- (iv) **Backpropagation:** Propagate the simulation results up the tree, updating value estimates for all visited nodes.

Adaptations for Imperfect Information: In imperfect-information games where states are partially observable, MCTS is adapted to work with information sets rather than specific states. An example is Information Set MCTS (ISMCTS), which uses random sampling over hidden states to guide decisions.

Mathematical Objective: MCTS seeks to maximize expected rewards. Let s be a state and a be an action. MCTS approximates the value $Q(s, a)$ by using simulated outcomes and updates action selection probabilities accordingly. While a closed-form solution is not always given, the principle is:

$$\max_a \mathbb{E}[\text{Return}(s, a)] \approx Q(s, a),$$

obtained through iterative simulations and updates.

Counterfactual Regret Minimization (CFR)

Counterfactual Regret Minimization (CFR) is a foundational method for solving extensive-form games. It operates on the principle of minimizing regret at each information set:

Core Principle: For each decision point in the game tree (an information set), CFR minimizes regret, defined as the difference between the payoff of the chosen action and the payoff of the best action in hindsight. Over iterations, regret values guide the update of strategy probabilities.

Mathematical Formulation: Let $R^t(a)$ denote the cumulative regret for action a at iteration t . If $u^{\text{cf}}(a)$ represents the counterfactual payoff for action a , then:

$$R^t(a) = R^{t-1}(a) + (u^{\text{cf}}(a) - u^{\text{cf}}(a^*)),$$

where $u^{\text{cf}}(a^*)$ is the best possible counterfactual payoff in hindsight at that iteration. The strategy probabilities are updated as:

$$\sigma^t(a) = \frac{\max(R^t(a), 0)}{\sum_{a'} \max(R^t(a'), 0)}.$$

Modern Variants: CFR+ (Tammelin, 2015) and Regret-Based Sampling (RBS) (Lanctot et al., 2009) accelerate convergence and focus computation on high-value regions of the game tree.

Algorithm 1 Pseudocode for CFR (adapted from Gibson et al.)

- 1: Initialize all regret values $R^0(a) = 0$ and strategies.
 - 2: **for** $t = 1$ to T **do**
 - 3: Compute strategies $\sigma^t(a)$ from $R^{t-1}(a)$ as above.
 - 4: For each information set, compute the counterfactual payoffs $u^{\text{cf}}(a)$ under σ^t .
 - 5: Update regrets: $R^t(a) = R^{t-1}(a) + (u^{\text{cf}}(a) - u^{\text{cf}}(a^*))$.
 - 6: The average strategy $\bar{\sigma}(a) = \frac{1}{T} \sum_{t=1}^T \sigma^t(a)$ approaches a Nash equilibrium as $T \rightarrow \infty$.
-

Recursive Belief-based Learning (ReBel)

Recursive Belief-based Learning (ReBel) is an advanced algorithm designed for imperfect-information games, integrating reinforcement learning (RL) with game-theoretic search:

Public Belief States: ReBel models belief states over hidden information. Let b denote a public belief state:

$$b = P(\text{hidden state} \mid \text{observed actions}).$$

These public belief states allow ReBel to reason effectively about unobserved aspects of the game.

Integration with RL and CFR: At any belief state b , ReBel combines:

- (i) RL updates to refine the policy $\pi(b, a)$.
- (ii) CFR-based regret minimization to ensure strategy convergence to Nash equilibria.

This hybrid approach balances exploration (via RL) and theoretical convergence guarantees (via CFR).

Algorithm 2 Pseudocode for the ReBel Algorithm (adapted from Brown et. al)

- 1: Initialize policy parameters π and value function approximations.
 - 2: Initialize public belief state model b .
 - 3: **for** each episode or iteration **do**
 - 4: Sample an initial belief state b_0 .
 - 5: Using CFR, compute a game-theoretic strategy for b_0 .
 - 6: Using RL, update $\pi(b, a)$ to improve performance given computed strategies.
 - 7: Propagate beliefs and strategies forward as actions are taken and new observations arrive, updating b and repeating the CFR + RL update process.
-

ReBel's integration of belief modeling, RL, and CFR-based updates enables it to scale to complex multiplayer imperfect-information games, such as multiplayer No-Limit Texas Hold'em poker.

Related Works

DeepStack and Libratus

DeepStack (Moravcik et al, 2017) and Libratus (Brown et al, 2017) are breakthrough algorithms for two-player No-Limit Texas Hold'em, employing depth-limited solving and strategic abstractions to compute near-optimal strategies. However, they are:

- Optimized primarily for two-player games.
- Heavily reliant on domain-specific abstractions that do not generalize well to multiplayer settings.
- Computationally intensive, especially in larger action spaces.

In contrast, ReBel and MCTS with CFR aim to extend these principles to multiplayer settings by leveraging public belief states for scalability.

Policy Gradient Methods

Policy gradient methods such as PPO (Shulman et al, 2017) and SAC (Haarnoja et al., 2018) are effective in continuous control tasks, optimizing policies directly using gradient-based updates. However:

- They struggle in extensive-form games like poker, where the strategy space is discrete and hidden information is critical.

Our approach retains the ability to reason over extensive action spaces and hidden states via CFR, which policy gradients do not directly achieve.

Search-Based Approaches

Search-based algorithms like Expectimax and AlphaZero (Silver et al., 2018) have demonstrated success in perfect-information games. Their limitations in imperfect-information settings are:

- Requiring fully observable states.
- Inability to directly handle hidden information.

By integrating MCTS with information set adaptations, we extend search-based frameworks to imperfect-information domains.

Regret Minimization Techniques

CFR+ (Tammelin, 2015) and RBS (Lancot et al., 2009) refine regret minimization, focusing on fast convergence and efficient exploration of the game tree. However:

- They do not inherently adapt to evolving strategies in real-time.

Our integration of CFR with RL (as in ReBel) and MCTS ensures that strategies adapt dynamically to the multiplayer environment.

Why ReBel and MCTS with CFR?

ReBel and MCTS with CFR effectively address the unique challenges of multiplayer imperfect-information games:

- **Scalability:** Public belief states in ReBel simplify maintaining consistent strategies across multiple players.

- **Flexibility:** CFR integrated with MCTS provides strategic depth while handling stochasticity and hidden information.

- **Adaptability:** RL ensures strategies evolve with changing gameplay dynamics, outperforming static solutions.

By combining RL and game-theoretic methods, our approach builds on foundational algorithms while overcoming their scalability and adaptability limitations in multiplayer scenarios.

Project Description

Our system models the problem of six-player No-Limit Texas Hold'em poker as a Reinforcement Learning (RL) task within the framework of a Markov Decision Process (MDP). Formally, an MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R)$, where:

State Space $\mathcal{S}$

The state space $\mathcal{S}$ represents a continuous encoding of the current environment. Each state $s \in \mathcal{S}$ includes:

- **Public information:** Visible to all players, such as community cards, the current pot size, and active bets.
- **Private information:** Each player's hole cards and hidden states known only to that player.

Action Space $\mathcal{A}$

The action space $\mathcal{A}$ comprises the finite set of permissible poker actions:

$$\mathcal{A} = \{\text{fold, call, check, bet}\}.$$

At each timestep t , the agent selects an action $a_t \in \mathcal{A}$ based on the observed state s_t .

Transition Probability Function P

The environment transitions from one state s_t to the next state s_{t+1} with probability:

$$P(s_{t+1} \mid s_t, a_t).$$

This transition depends on both the agent's action and the stochastic nature of the game. For instance, the probability distribution accounts for the random dealing of community cards, as well as the actions chosen by other players.

Reward Function R

The reward function $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ assigns a scalar reward to state-action pairs. Formally:

$$r_t = R(s_t, a_t),$$

where r_t reflects the monetary outcome of performing action a_t in state s_t . Positive rewards correspond to winnings, while negative rewards represent losses or penalties for sub-optimal decisions.

Policy $\pi(a \mid s)$

A policy $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ maps states to probability distributions over actions. The agent’s goal is to find a policy $\pi(a \mid s)$ that maximizes the expected cumulative discounted reward:

$$\mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r_k \mid s_0, \pi \right],$$

where $\gamma \in [0, 1]$ is the discount factor balancing the importance of immediate and future rewards.

Approach

Combining Recursive Belief-based Learning (ReBeL) and Counterfactual Regret Minimization (CFR) provides a rigorous game-theoretic foundation for policy computation. Monte Carlo Tree Search (MCTS) is then employed to efficiently explore and evaluate actions under imperfect information, guiding the agent toward strategies that yield higher expected returns in multi-player No-Limit Texas Hold’em.

This integrated approach allows for fine-grained strategy computation and leverages the strengths of both RL and game-theoretic methods. By iterating this process across multiple rounds and exploiting public belief states, our system refines its strategies and improves its performance against highly dynamic opponents.

ReBeL Implementation

Our ReBeL agent builds on the Recursive Belief-based Learning framework, combining reinforcement learning, regret minimization, and neural network-based function approximation to address the challenges of multiplayer No-Limit Texas Hold’em poker.

State Representation

The state representation integrates player and environment information into a single vector using a `featurize_state` function:

- **Player Hand Features:** Hand strength and relative rank are computed using a `HandEvaluator` module, which provides detailed metrics such as the likelihood of achieving specific poker hands (e.g., flush, straight, full house) and normalized hand strengths.
- **Opponent Metrics:** Betting patterns and behavioral statistics for each opponent are encoded, including metrics such as bet averages, aggression frequencies, and betting volatility.
- **Final State Vector:** The combined vector includes 21 features for the player’s hand and 8 features per opponent, enabling the agent to reason about both private information and opponents’ tendencies.

Reward Modification

The reward function is based on the outcome of a round rather than the outcome of a single hand, encouraging the agent to strategize for long-term profitability rather than immediate gains. Formally:

$$R(\text{Win}) = 1 \quad \text{if Win} = (\text{profit} = \max(\text{profit for all players}))$$

$$R(\text{Loss}) = -1 \quad \text{if Loss} = (\text{profit} < 0)$$

$$R(\text{Share}) = \frac{\text{profit}}{\max(\text{profit for all players})} \quad \text{if } (\text{profit} \geq 0 \text{ and } \text{profit} < \max)$$

Here,

$$\text{profit} = \text{final stack} - \text{starting stack}.$$

This reward structure encourages maximizing overall profitability across a round and penalizes losses. By focusing on cumulative outcomes, the model is incentivized to adopt strategies that perform well in diverse scenarios, aligning with the long-term goals of poker gameplay.

During the hand, the agent receives zero reward for every transition; the above rewards are assigned only at the end of the round. Developing a robust in-hand reward function is an area for future improvement, as intermediate rewards could guide the agent to navigate hands more effectively.

Key Components

CFR-Based Subgame Solving:

- Uses a depth-limited variant of CFR (CFR-D) to solve subgames.
- Regret values and strategies are iteratively updated over multiple iterations to approach equilibrium policies.
- A running average of strategies and value estimates is maintained for each information state, ensuring convergence.

Neural Networks for Policy and Value Estimation:

- Three neural networks approximate value functions and action probabilities.
- **Value Network:** Predicts the expected value of a state.
- **Policy Network:** Outputs probabilities for each poker action (fold, call, check, or bet).
- **Bet Value Network:** Estimates optimal bet sizes within valid ranges, using a softplus activation to ensure non-negative outputs.
- All networks utilize GELU activations and layer normalization for stability and efficient learning.

Public Belief State (PBS) Management:

- PBS aggregates observed information (e.g., betting history, community cards) to maintain consistent probabilistic reasoning about hidden information.
- PBS is updated dynamically as actions and outcomes are observed, providing a robust framework for decision-making in imperfect-information environments.

Experience Replay and Training:

- A replay buffer stores transitions, enabling batch training for both policy and value networks.
- Loss functions incorporate CFR-based targets and temporal difference (TD) learning, balancing short-term exploitation with long-term strategic planning.

Decision-Making Pipeline

1. **State Encoding:** Encode the current game state into a feature vector using hand strength, opponent metrics, and PBS updates.
2. **Subgame Solving:** Perform depth-limited CFR-D to compute equilibrium strategies for the current game state.
3. **Action Selection:** Sample action probabilities from the policy network, incorporating exploration (e.g., ϵ -greedy). If betting is required, the bet size is predicted by the Bet Value Network.
4. **Policy and Value Updates:** After each game iteration, update strategies and values using CFR-based regret matching and neural network optimization.

Monte Carlo Counterfactual Regret Minimization Implementation

Our implementation combines three key algorithmic approaches: MCTS, CFR, and MCCFR.

Monte Carlo Tree Search Foundation

MCTS operates in four phases:

Selection: From the root node, select child nodes using the UCB1 formula:

$$UCB1(v) = \bar{X} + C\sqrt{\frac{\ln(N)}{n}}$$

where: - $\bar{X}$: average reward - N : total parent visits - n : child node visits - C : exploration constant (typically $\sqrt{2}$)

Expansion: Add new nodes to explore unseen game states. Each node v stores:

$N(v), Q(v), P(v)$, parent and child references

Simulation: Random playouts estimate state values:

$$V(s) = \sum_t \gamma^t r(s_t, a_t)$$

where γ is the discount factor and $r(s_t, a_t)$ the immediate reward.

Backpropagation: Update node statistics:

$$N(v) = N(v) + 1$$

$$Q(v) = Q(v) + \frac{r - Q(v)}{N(v)}$$

Counterfactual Regret Minimization

CFR adjusts strategies based on regret:

Strategy Computation: For an information set I :

$$\sigma_i(I, a) = \frac{R_i^+(I, a)}{\sum_{a'} R_i^+(I, a')}$$

where $R_i^+(I, a) = \max(R_i(I, a), 0)$ is the positive regret.

Counterfactual Value:

$$v(I, \sigma) = \sum_h \pi_{-i}(h) \sigma_i(h) u(h)$$

Regret Updates: After each iteration:

$$R_i(I, a) := R_i(I, a) + v_i(I, a \rightarrow \sigma) - v_i(I, \sigma)$$

Monte Carlo CFR Integration (MCCFR)

MCCFR merges MCTS sampling and CFR's regret updates:

Information Set Management: Each information set I tracks cumulative regret, cumulative strategy, visits, and children.

Strategy Computation with Exploration:

$$\sigma(I, a) = (1 - \epsilon) \cdot \frac{R^+(I, a)}{\sum_{a'} R^+(I, a')} + \frac{\epsilon}{|A|}$$

where $\epsilon = 0.05$ and $|A|$ is the number of legal actions.

Value Estimation:

$$V(I) = r + \gamma \sum_a \sigma(I, a) Q(I, a)$$

Update Mechanism: Updates integrate MCTS statistics and CFR regrets. Thresholding and scaling ensure numerical stability.

Implementation Details for Poker

Domain-specific optimizations for No-Limit Texas Hold'em:

State Representation:

```
def get_state_key(self, observation):
    return (
        position,           # Player position
        pot//100,           # Pot size
        street,             # Game street
        tuple(hole_cards),  # Private cards
        tuple(community)   # Community cards
    )
```

Action Space (Bet Sizing):

```
def get_bet_size(self, legal_range, confidence):
    pot = self.observation[indices.POT_SIZE]
    min_bet, max_bet = legal_range
    scale = 0.5 + 0.5 * confidence
    return min(max_bet, max(min_bet, pot * scale))
```

Reward Calculation:

```
def get_reward(self, stack_change, pot_size):
    return stack_change / max(pot_size, 1)
```

Performance Optimizations

- **Memory Management:** Periodic pruning, efficient encoding.
- **Sampling Efficiency:** Early stopping, progressive widening, action abstraction.
- **Numerical Stability:** Thresholded updates ($\tau = 1000$), strategy normalization, reward scaling.

This MCCFR framework balances exploration and exploitation, efficiently handling large action spaces and imperfect information in a multi-player setting.

Experiments

ReBeL Experiments

To evaluate the performance of our improved ReBeL agent in a multiplayer No-Limit Texas Hold'em environment, we conducted two sets of experiments. These trials tested the agent's ability to learn and adapt against a variety of opponent strategies over 1,000 rounds per trial.

Experimental Setup The environment was configured for a 6-player No-Limit Texas Hold'em setting. ReBeL faced the following types of agents:

- **Aggressive Agent:** Bets all-in on every action.
- **Fold Agent:** Folds every hand without contest.
- **Q-Learning Agent:** Uses tabular Q-learning to optimize strategies.
- **Determined Agent:** Never folds, always calling or raising.
- **Random Agent:** Takes random actions without any strategy.

Each trial consisted of 1,000 rounds, and we conducted 10 trials for each experimental configuration. Performance was measured by the cumulative chip count at the end of each trial.

Results:

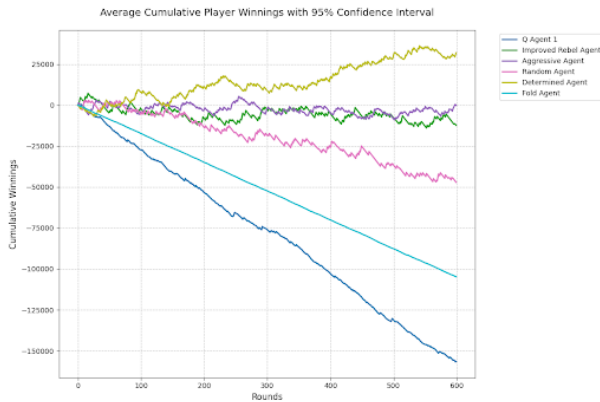


Figure 1: ReBeL vs Diverse Agents

Experiment 1: ReBeL vs. Diverse Agent Pool In the first experiment, ReBeL competed against a set of five distinct agents (Aggressive, Fold, Q-Learning, Determined, and Random). Key observations include:

- **Performance:** ReBeL performed moderately, especially when facing Determined and Aggressive agents.
- **Observations:** Suboptimal results were attributed to a known issue in the environment, where small pots were not distributed correctly. Ideally, the sum of winnings across all agents should be zero, but the flawed distribution allowed Determined and Aggressive agents to exploit this mechanic by consistently going all-in.
- **Conclusion:** The environment's inaccuracies heavily influenced the results. Although ReBeL's performance was not as strong as desired, it did demonstrate some learning advantages over Q-Learning, even under flawed conditions.

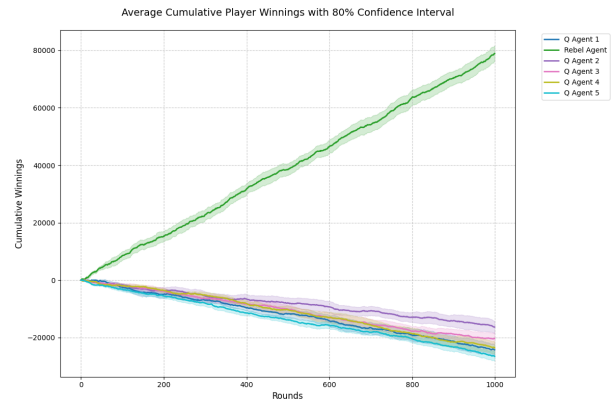


Figure 2: ReBeL vs Q-Learning Agents

Experiment 2: ReBeL vs. Q-Learning Agents In the second experiment, ReBeL faced a field of five Q-Learning agents, each independently optimizing its strategy. The outcomes differed significantly:

- **Performance:** ReBeL consistently outperformed all Q-Learning agents, achieving a substantial lead in cumulative chip count across all 10 trials.
- **Analysis:** The superior performance of ReBeL highlights its ability to learn and adapt more effectively than Q-Learning. By utilizing public belief states and incorporating regret-based strategy refinement, ReBeL exploited weaknesses in the Q-Learning agents' strategies.
- **Observation:** Without the Aggressive agent forcing frequent all-ins, the pot distributions were more consistent, allowing ReBeL to realize its full potential in a properly functioning environment.

Discussion The contrasting results from the two experiments underscore several important points:

- **Importance of Environment Fidelity:** The pot distribution issues in Experiment 1 highlight the critical need for accurate and fair environmental dynamics. Flaws can skew results and mask the true capabilities of an algorithm.
- **ReBeL's Strengths:** In Experiment 2, ReBeL's dynamic modeling of the environment and continuous strategy refinement allowed it to surpass Q-Learning agents, confirming its suitability for imperfect-information games.
- **Limitations:** The reliance on a flawed environment in Experiment 1 demonstrates the challenges in benchmarking advanced algorithms like ReBeL. The quality of the environment directly influences the validity and interpretability of experimental outcomes.

Monte Carlo Counterfactual Regret Minimization Experiments

Experimental Setup The environment was configured for a 6-player No-Limit Texas Hold'em game, with ReBeL facing various types of agents:

- **Aggressive Agent:** Bets all-in on every action.
- **Fold Agent:** Folds every hand without contest.
- **Q-Learning Agent:** Uses tabular Q-learning to optimize strategies.
- **Determined Agent:** Never folds, always calling or raising every bet.
- **Random Agent:** Takes random actions without any strategy.

Each trial consisted of 1,000 rounds, and we conducted 10 trials for each experimental configuration. Performance was measured by the cumulative chip count at the end of each trial.

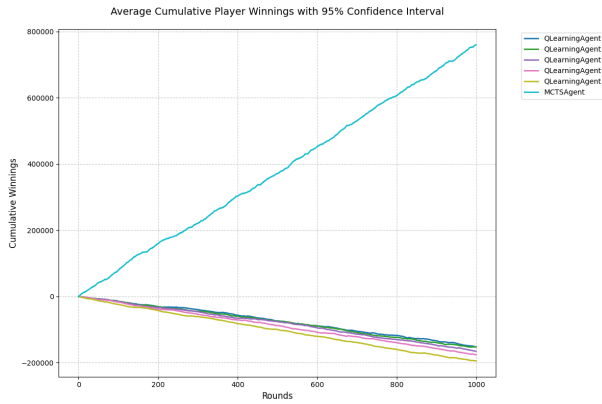


Figure 3: MCTS vs Q-Learning Agents

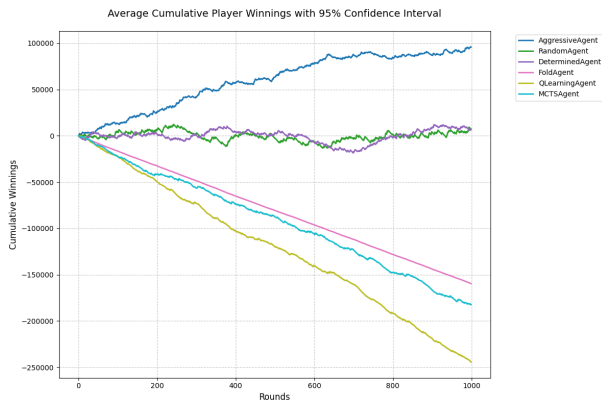


Figure 4: MCTS vs Assorted Agents

MCTS Experiment

Analysis of MCTS Results: Against Q-Learning agents (Figure 3), MCTS shows moderate performance. While it can outmaneuver simple Q-Learning strategies initially, its performance plateaus over time as the game tree grows large, reflecting computational overhead and less efficient deep exploration. Against the mixed pool of agents (Figure 4),

MCTS demonstrates some adaptability but struggles with highly aggressive or complex strategies. This suggests that while MCTS provides a strong baseline for exploration, it may require enhancements, such as action abstraction or pruning, to maintain performance in more demanding scenarios.

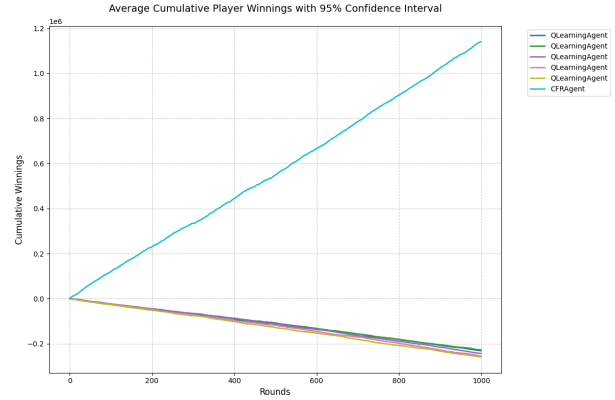


Figure 5: CFR vs Q-Learning Agents

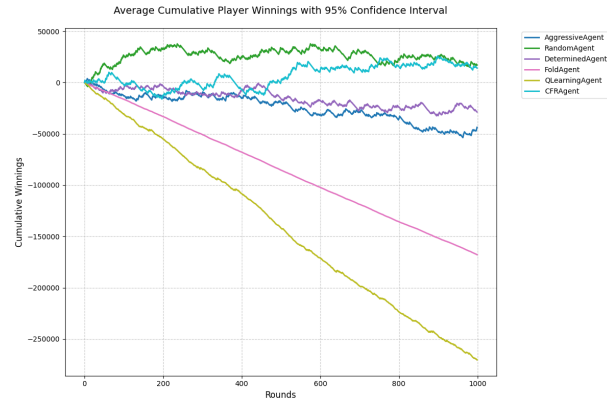


Figure 6: CFR vs Assorted Agents

CFR Experiment

Analysis of CFR Results: CFR (Figures 5 and 6) consistently outperforms Q-Learning by leveraging regret minimization to refine strategies and identify profitable actions over time. While it handles simpler scenarios well and converges effectively in two-player settings, the complexity of no-limit betting and a diverse opponent mix introduces challenges. Although CFR outperforms naive agents, it struggles to fully capitalize against aggressive or highly variable opponents. This indicates the need for enhanced abstractions or integration with other techniques to handle the large state and action spaces in multiplayer poker.

MCCFR Experiment

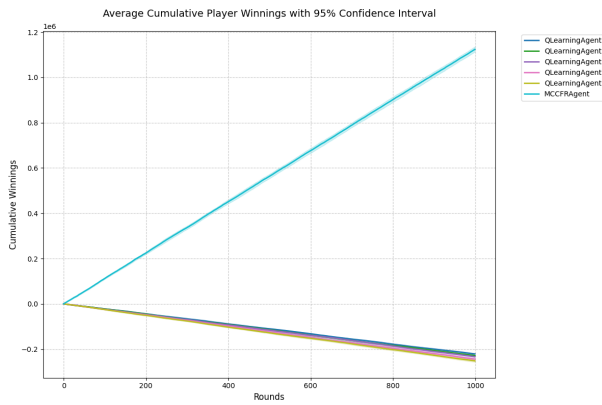


Figure 7: MCCFR vs Q-Learning Agents

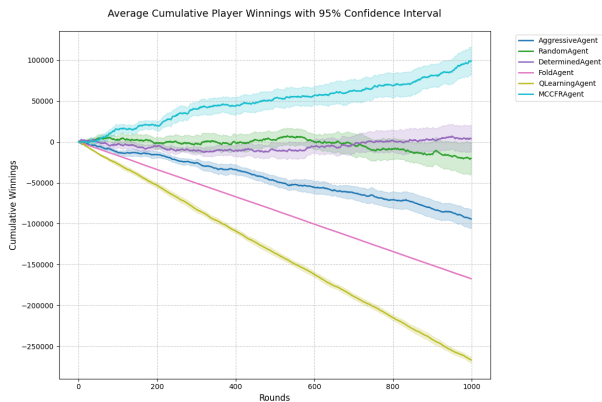


Figure 8: MCCFR vs Assorted Agents

Analysis of MCCFR Results: As shown in Figures 7 and 8, MCCFR integrates the strengths of MCTS and CFR. Against Q-Learning agents, it not only surpasses them but shows a clear upward trend in cumulative winnings, indicating robust long-term strategy refinement. When facing a diverse set of opponents, MCCFR maintains balanced performance, exploiting conservative players, adapting to random strategies, and resisting aggressive tactics more effectively than MCTS or pure CFR alone. However, there is still some room for improvement in handling extreme aggression, and computational efficiency remains a consideration.

Discussion

Performance Against Q-Learning Agents: In experiments against multiple Q-Learning agents, both CFR and MCCFR demonstrated superior performance, consistently accumulating positive winnings while Q-Learning agents showed a steady decline. This success can be attributed to:

- **Better Strategy Adaptation:** CFR and MCCFR's ability to minimize regret across different scenarios allows them to develop more robust strategies compared to Q-Learning's simpler value-based approach.
- **Efficient Exploration:** The sampling mechanisms in

MCCFR enable more effective exploration of the large action space, particularly in betting decisions, compared to Q-Learning's ϵ -greedy approach.

- **Long-term Strategy Development:** The cumulative winnings show a consistent upward trend, indicating that both CFR and MCCFR successfully develop and refine effective strategies over time, while Q-Learning agents struggle to find stable policies.

Performance Against Mixed Agent Types: When tested against a diverse set of agents (Aggressive, Random, Determined, Fold, and Q-Learning), our implementations showed varying levels of success:

- **Against Aggressive Agents:** MCTS and MCCFR demonstrated resilience against aggressive betting strategies, performing better than Q-Learning but still leaving room for improvement.
- **Against Passive Agents:** Both implementations significantly outperformed the Fold agent, exploiting conservative strategies and adjusting bet sizing effectively.
- **Overall Adaptation:** MCCFR demonstrated the most balanced performance across different opponent types, successfully maintaining positive winnings and generalizing better than pure MCTS or CFR approaches.

Implementation-Specific Analysis:

- **MCTS:** Showed initial promise but was computationally expensive and less efficient in deeper game trees.
- **CFR:** Provided strong theoretical guarantees and robust convergence but faced challenges with large, no-limit betting spaces.
- **MCCFR:** Combined MCTS exploration with CFR's regret minimization, improving exploration, handling of no-limit betting, and overall adaptability.

Limitations and Challenges:

- **Computational Resources:** High computational and memory demands limit scalability.
- **Strategy Adaptation:** Delays in adapting to dramatic strategy changes indicate potential for improved dynamic strategy updates.
- **Scalability:** Larger player counts and longer games require more efficient state representations and pruning strategies.

Future Improvements: Potential enhancements include:

- **Enhanced Pruning and Abstraction:** More sophisticated tree pruning and action abstraction to handle no-limit betting efficiently.
- **Hybrid Approaches:** Integrating neural networks for value estimation, improved sampling strategies for MCCFR, and refined opponent modeling.

- **Performance Optimization:** Parallelizing computations and better memory management to handle more complex scenarios.

Overall, these findings highlight that while MCTS, CFR, and MCCFR show promise, particularly MCCFR in adapting to a range of opponents, there remains significant scope for optimization in terms of scalability, efficiency, and handling of extreme play styles.

Conclusion

This project explored the application of Recursive Belief-based Learning (ReBeL) in a 6-player No-Limit Texas Hold'em poker environment, demonstrating its adaptability and improved performance over Q-Learning agents. ReBeL leveraged reinforcement learning, regret minimization, and public belief states to navigate complex, imperfect-information scenarios. While it outperformed simpler RL methods, its struggles against aggressive deterministic agents underscored limitations in both the experimental environment and the state representation.

In addition, we investigated Monte Carlo Counterfactual Regret Minimization (MCCFR) experiments integrating MCTS and CFR techniques. Although MCCFR showed marginal improvements in handling diverse and aggressive opponents compared to pure CFR or MCTS approaches, these gains were not as pronounced as hoped. The approach still faced scalability and computational challenges, indicating that further optimizations, better abstractions, and more refined strategy adaptations are needed to fully exploit its potential.

Improvements and Limitations

Several challenges affected the quality and interpretability of our results:

Environment Issues: The flawed handling of small pot distributions enabled certain agents to exploit the environment, skewing results. Addressing these environmental quirks is vital to ensure fair and accurate evaluations of advanced RL methods in future studies.

State Representation: While ReBeL effectively employed public belief states, more granular features such as detailed opponent behavior tracking, stack sizes, and betting patterns could enhance strategic decision-making. Introducing specialized networks for hand confidence and opponent strength predictions may further refine strategy quality.

Computational Complexity: Running ReBeL and MCCFR is computationally expensive, especially for longer simulations or more complex action abstractions. Techniques like error clipping, optimized batch processing, and parallelized computations using multiple CPUs or GPUs could help alleviate these bottlenecks and improve scalability.

Key Takeaways

Our research provided insights into the challenges of developing competitive poker agents under imperfect-information conditions:

- **ReBeL's Strengths:** Integration of RL, CFR-based reasoning, and belief-state modeling allowed ReBeL to outperform baseline RL agents.
- **MCCFR's Potential and Limits:** While integrating MCTS and CFR into MCCFR provided marginal improvements over standalone methods, the complexity of no-limit betting and multi-agent interactions limited its impact without further refinement.
- **Room for Growth:** Improving environment fidelity, enhancing state representations, and increasing computational efficiency are critical steps toward unlocking the full potential of these advanced RL techniques.

This work lays the foundation for further exploration of advanced RL methods in imperfect-information settings. With more robust environments, richer features, and efficiency-focused optimizations, techniques like ReBeL and MCCFR can be further developed, contributing to the broader field of multi-agent reinforcement learning.

Other Takeaways

Our initial attempt to build a movie recommendation system using RL was hampered by a small state space and limited data, preventing effective learning. By contrast, Texas Hold'em presented a vast and complex continuous state space, revealing new opportunities and challenges. However, environmental flaws and handling of aggressive agents introduced exploitable dynamics that influenced outcomes. This experience emphasized the importance of well-designed, high-fidelity environments and thoughtful feature engineering in achieving robust RL performance, particularly in multiplayer, imperfect-information domains like poker.

References

- Brown, N., Bakhtin, A., Lerer, A., and Pineau, J. 2020. "ReBeL: Scaling Self-Play Reinforcement Learning to Large Imperfect-Information Games." *arXiv preprint arXiv:2007.13544*.
- Gibson, R.; Burch, N.; Lanctot, M.; and Szafron, D. 2012. "Efficient Monte Carlo Counterfactual Regret Minimization in Games with Many Player Actions." *Advances in Neural Information Processing Systems*, 25: 190–198.
- Haarnoja, T.; Zhou, A.; Abbeel, P.; and Levine, S. 2018. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *arXiv preprint arXiv:1801.01290*. <https://arxiv.org/abs/1801.01290>.
- Lanctot, M.; Waugh, K.; Zinkevich, M.; and Bowling, M. 2009. Monte Carlo Sampling for Regret Minimization in Extensive Games. *Advances in Neural Information Processing Systems 22: Proceedings of the 2009 Conference*, 1078–1086.
- Moravčík, M.; Schmid, M.; Burch, N.; Lisý, V.; Morrill, D.; Bard, N.; Davis, T.; Waugh, K.; Johanson, M.; and Bowling, M. 2017. DeepStack: Expert-level artificial intelligence in heads-up no-limit poker. *Science*, 356(6337): 508–513. <https://doi.org/10.1126/science.aam6960>.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347. <https://arxiv.org/abs/1707.06347>.

Silver, D.; Hubert, T.; Schrittwieser, J.; Antonoglou, I.; Lai, M.; Guez, A.; Lanctot, M.; Sifre, L.; Kumaran, D.; Graepel, T.; Lillicrap, T.; Simonyan, K.; and Hassabis, D. 2017. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. arXiv preprint arXiv:1712.01815. <https://arxiv.org/abs/1712.01815>.

Tammelin, O. 2014. Solving Large Imperfect Information Games Using CFR+. arXiv preprint arXiv:1407.5042. <https://arxiv.org/abs/1407.5042>.